EXPERIMENT 4

Aim: Hands on Solidity Programming Assignments for creating Smart Contracts

Theory:

1. Primitive Data Types, Variables and Functions

Solidity provides basic data types that are used to build smart contracts. Common primitive types include uint and int for unsigned and signed integers, bool for logical values, address for storing Ethereum account addresses, and string or bytes for textual and binary data. These types form the core building blocks of contract logic.

Variables in Solidity are classified based on where and how they are stored:

- **State variables** are declared inside a contract but outside functions and are permanently stored on the blockchain.
- **Local variables** exist only during the execution of a function.
- **Global variables** such as msg.sender, msg.value, block.timestamp, and block.number provide information about the transaction and blockchain environment.

Functions define the behavior of a contract. Solidity introduces special function types:

- **pure functions** neither read nor modify the blockchain state.
- **view functions** read state variables but do not modify them.
- Regular functions can modify state and therefore consume gas when executed through transactions.

2. Function Inputs and Outputs

Functions can accept parameters and return values. Inputs allow users or other contracts to pass data into a function, while outputs provide results after computation. Solidity supports multiple return values and named return variables, improving clarity and readability.

For example, a function can return multiple values such as an integer and a boolean. Solidity also allows destructuring assignments, where returned values are unpacked directly into variables. Proper use of inputs and outputs makes contracts more modular and reusable.

3. Visibility, Modifiers and Constructors

Visibility specifiers control access to functions and state variables:

- public – accessible inside and outside the contract.
- private – accessible only within the contract.
- internal – accessible within the contract and its derived contracts.
- external – callable only from outside the contract.

Modifiers are reusable pieces of logic that alter the behavior of functions. They are commonly used for access control (e.g., restricting certain actions to the contract owner) and for enforcing conditions before execution.

4. Control Flow Statements

Solidity supports conditional and iterative control structures similar to other programming languages:

- **if, else if, else** statements allow decision-making based on conditions.
- **for, while, and do-while loops** enable repeated execution of code.

However, loops must be used carefully in Solidity because each iteration consumes gas. Excessive iterations can make transactions expensive or even cause them to fail due to running out of gas.

5. Data Structures: Arrays, Mappings, Structs and Enums

Solidity provides several data structures to manage and organize data:

- **Arrays** store ordered lists of elements. They can be fixed-size or dynamic.
- **Mappings** store key-value pairs and allow efficient lookup of values using a key. Unlike arrays, mappings cannot be iterated over directly.
- **Structs** allow grouping related data into a custom data type, improving code organization.
- **Enums** define a set of predefined constant values, making state representation clearer and more readable.

These structures are essential for building real-world decentralized applications where data organization is critical.

6. Data Locations

Solidity requires explicit specification of data locations for reference types:

- **storage**: Permanent storage on the blockchain (used for state variables).
- **memory**: Temporary storage used during function execution.

- **calldata**: Read-only temporary storage for external function parameters.

Assignments between these locations behave differently. Storage references point to the same data, while assignments between storage and memory create copies. Choosing the correct data location affects gas cost and performance.

7. Transactions, Ether and Gas

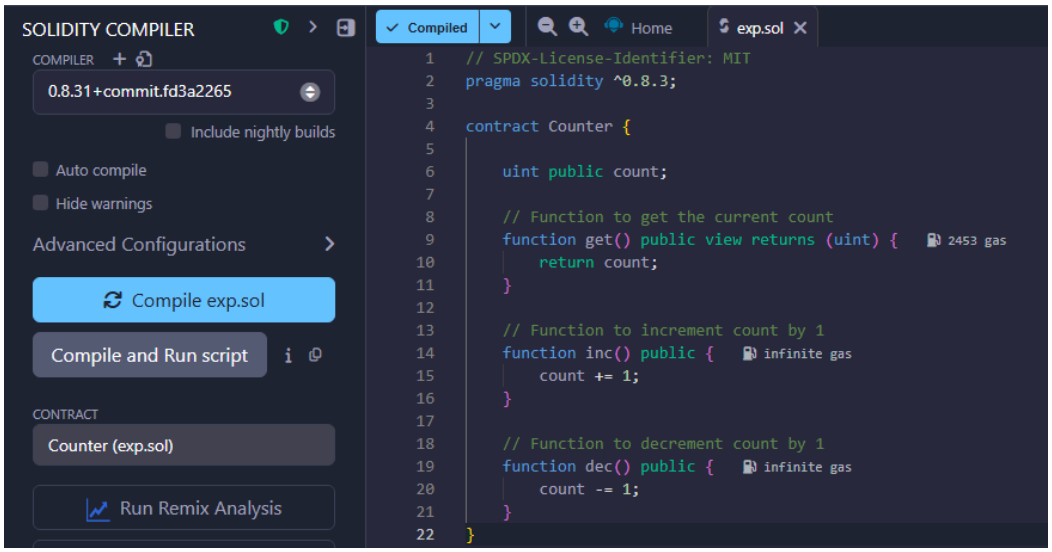
Ethereum transactions are used to interact with contracts or transfer Ether.

- **Ether and Wei**: Ether is the native cryptocurrency of Ethereum. All values are internally represented in Wei (1 Ether = 10^{18} Wei). Units like gwei are commonly used to express gas prices.
- **Gas and Gas Price**: Gas measures computational work required to execute operations. The total transaction cost equals gas used multiplied by gas price. Efficient contract design helps reduce gas consumption.
- **Sending and Receiving Ether**: Contracts can receive Ether using payable functions and special functions like receive() or fallback(). Ether can be sent using call, which is currently recommended due to flexibility and safety when implemented correctly with proper checks.

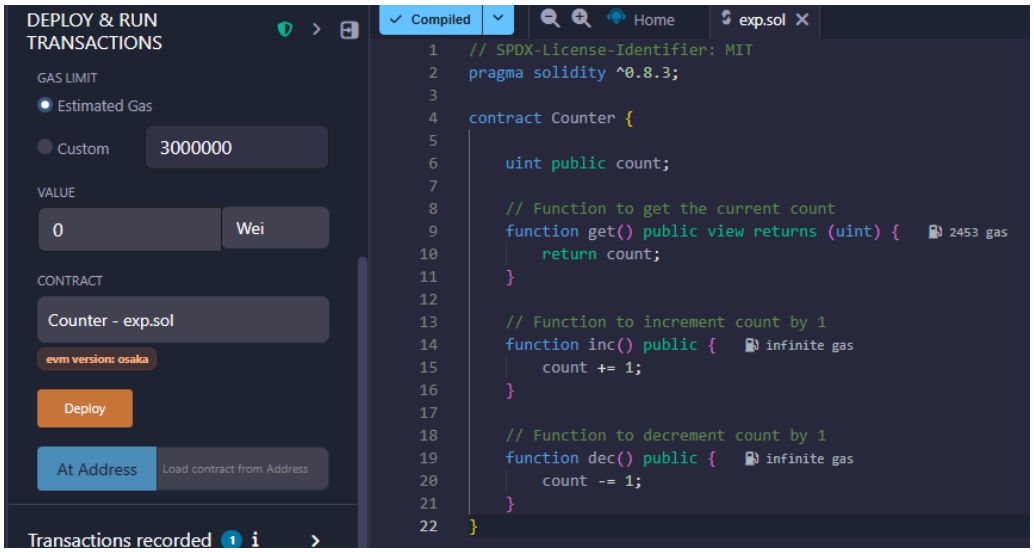
Implementation:

Tutorial 1:

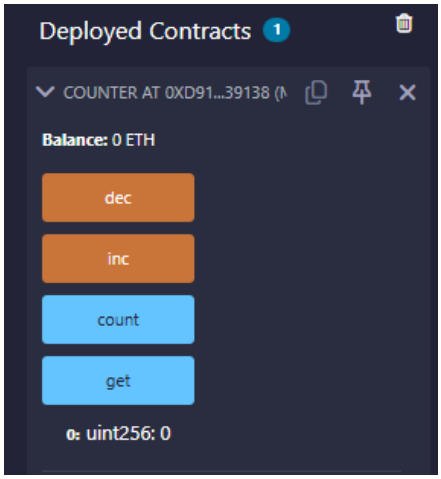
Compile code



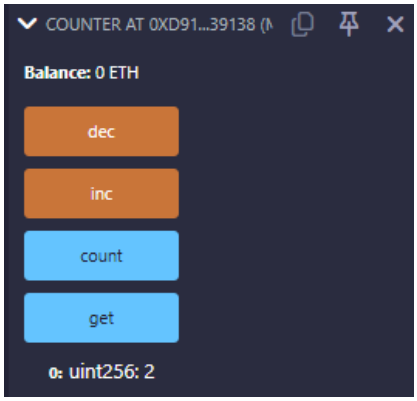
Deploy:



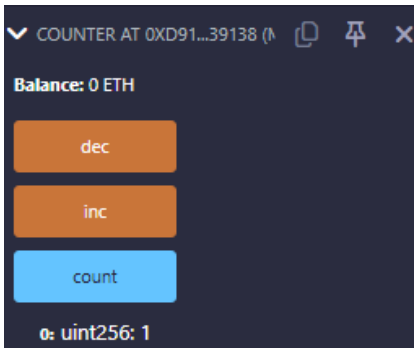
Get:



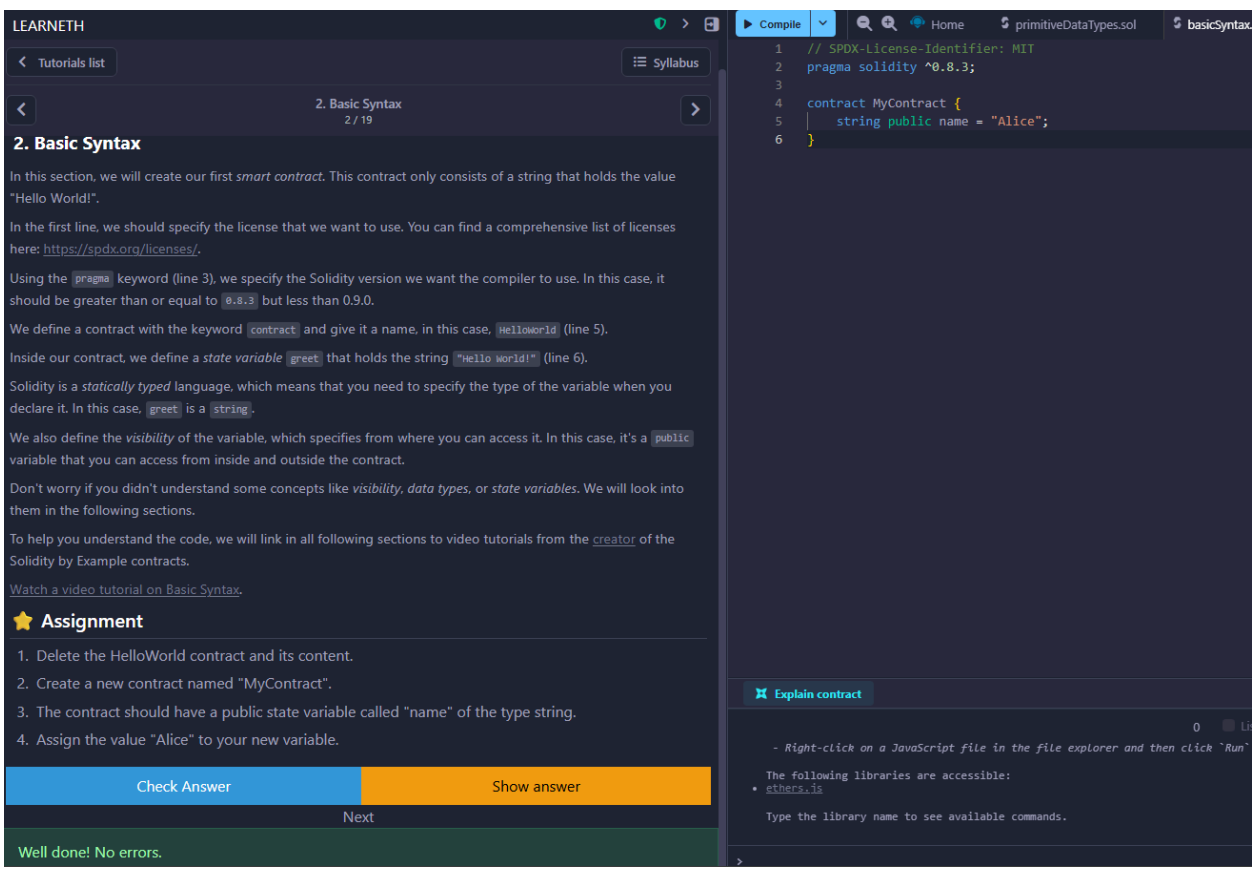
Increment:



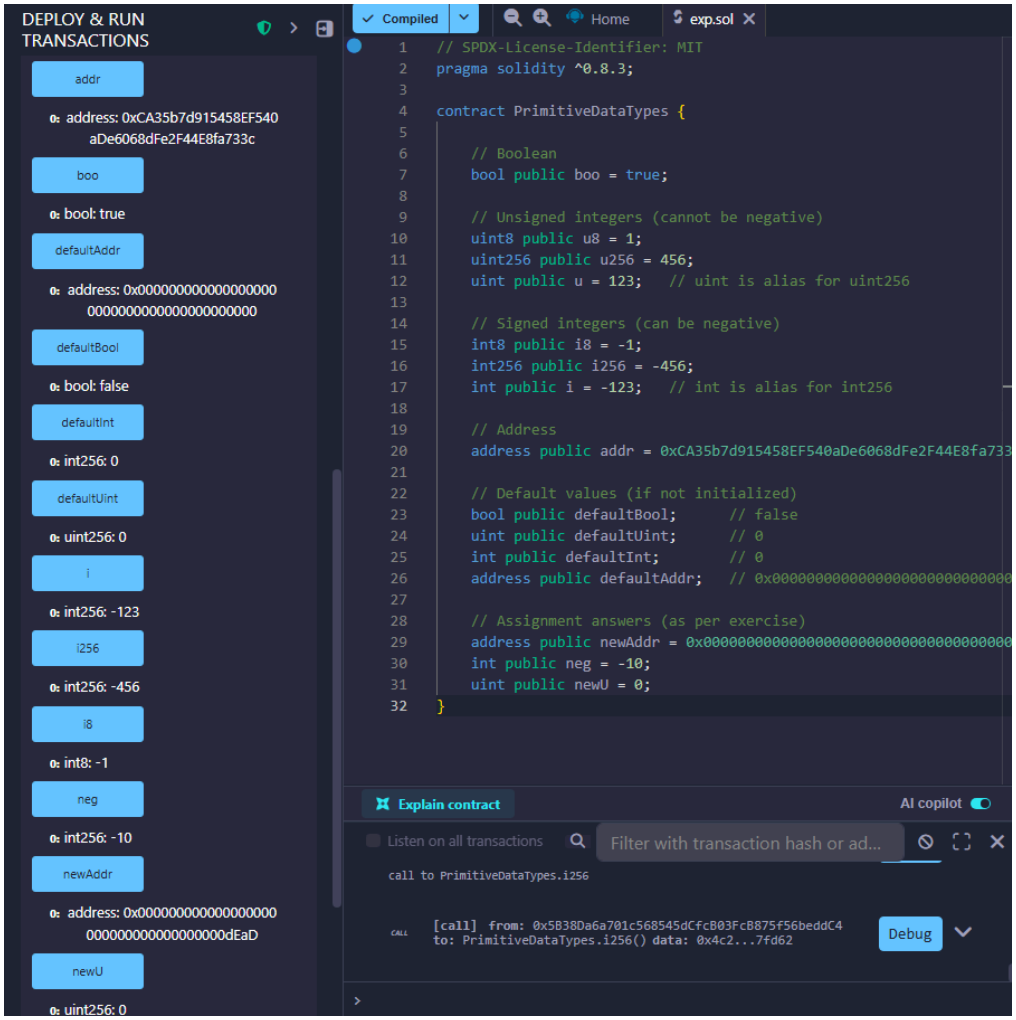
Decrement:



Tutorial 2:



Tutorial 3:


DEPLOY & RUN TRANSACTIONS
addr
0: address: 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733c
boo
0: bool: true
defaultAddr
0: address: 0x0000000000000000000000000000000000000000
defaultBool
0: bool: false
defaultInt
0: int256: 0
defaultUint
0: uint256: 0
i
0: int256: -123
i256
0: int256: -456
i8
0: int8: -1
neg
0: int256: -10
newAddr
0: address: 0x000000000000000000000000000000000000dEaD
newU
0: uint256: 0
Compiled
Home
exp.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.3;
contract PrimitiveDataTypes {
// Boolean
bool public boo = true;
// Unsigned integers (cannot be negative)
uint8 public u8 = 1;
uint256 public u256 = 456;
uint public u = 123; // uint is alias for uint256
// Signed integers (can be negative)
int8 public i8 = -1;
int256 public i256 = -456;
int public i = -123; // int is alias for int256
// Address
address public addr = 0xCA35b7d915458EF540aDe6068dFe2F44E8fa733
// Default values (if not initialized)
bool public defaultBool; // false
uint public defaultUint; // 0
int public defaultInt; // 0
address public defaultAddr; // 0x0000000000000000000000000000
// Assignment answers (as per exercise)
address public newAddr = 0x0000000000000000000000000000000000000
int public neg = -10;
uint public newU = 0;
}
Explain contract
AI copilot
Listen on all transactions
Filter with transaction hash or ad...
call to PrimitiveDataTypes.i256
CALL
[call] from: 0x5B38Da6a701c568545dCfcB03FcB875f56beddC4
to: PrimitiveDataTypes.i256() data: 0x4c2...7fd62
Debug

Tutorial 4:

LEARNETH

Tutorials list | Syllabus

4. Variables
4 / 19

Global Variables are used to retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Watch video tutorials on State Variables, Local Variables, and Global Variables.

Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer

Show answer

Next

Well done! No errors.

Compile | primitiveDataTypes.sol | basicSyntax.sol | variables.s

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Variables {
5
6      string public text = "Hello";
7      uint public num = 123;
8
9      // ⭐ Assignment variable
10     uint public blockNumber;
11
12     function doSomething() public {    22312 gas
13         uint i = 456; // local variable
14
15         // Assign current block number to state variable
16         blockNumber = block.number;
17     }
18 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 5:

LEARNETH

Tutorials list | Syllabus

5.1 Functions - Reading and Writing to a State Variable
5 / 19

function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on Functions.

Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer

Show answer

Next

Well done! No errors.

Compile | basicSyntax.sol | variables.sol 1 | readAndWrite.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract SimpleStorage {
5
6      uint public num;
7
8      // ⭐ Assignment
9      bool public b = true;
10
11     function set(uint _num) public {    22536 gas
12         num = _num;
13     }
14
15     function get() public view returns (uint) {    2475 gas
16         return num;
17     }
18
19     // ⭐ Assignment function
20     function get_b() public view returns (bool) {    2539 gas
21         return b;
22     }
23 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 6:

LEARNETH

Tutorials list | Syllabus

5.2 Functions - View and Pure
6 / 19

contains certain opcodes."

From the Solidity documentation.

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions view and pure can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

⭐ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

Check Answer

Show answer

Next

Well done! No errors.

Compile | .sol | viewAndPure.sol | modifiersAndConstructors.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract ViewAndPure {
5
6      uint public x = 1;
7
8      function addToX(uint y) public view returns (uint) {    infinite
9          return x + y;
10     }
11
12     function add(uint i, uint j) public pure returns (uint) {    in
13         return i + j;
14     }
15
16     // ⭐ Assignment
17     function addToX2(uint y) public {    infinite gas
18         x = x + y;
19     }
20 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

```
 - Right-click on a JavaScript file in the file explorer and then click
`Run`

The following libraries are accessible:
• ethers.js

Type the library name to see available commands.

>
```

Tutorial 7:

LEARNETH

Tutorials list | Syllabus

5.3 Functions - Modifiers and Constructors
7 / 19

when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that x can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

Check Answer

Show answer

Next

Well done! No errors.

modifiersAndConstructors.sol

```
38          _;
39      }
40
41      modifier increaseXbyY(uint y) {
42          _;
43          x = x + y;
44      }
45
46      // ⭐ Assignment function (body must remain empty)
47      function increaseX(uint y)    infinite gas
48          public
49          onlyOwner
50          biggerThan0(y)
51          increaseXbyY(y)
52      {
53      }
54
55      // Reentrancy protection modifier
56      modifier noReentrancy() {
57          require(!locked, "No reentrancy");
58
59          locked = true;
60          _;
61          locked = false;
62      }
63
64      function decrement(uint i) public noReentrancy {    infinite ga
65          x -= i;
66
67          if (i > 1) {
68              decrement(i - 1);
69          }
70      }
71  }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 8:

LEARNETH

Tutorials list | Syllabus

5.4 Functions - Inputs and Outputs
8 / 19

Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract functions.

"*[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible.*" From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

Assignment

Create a new function called `returnTwo` that returns the values `-2` and `true` without using a return statement.

Check Answer

Show answer

Next

Well done! No errors.

inputsAndOutputs.sol

```
58          )
59      {
60          (uint i, bool b, uint j) = returnMany();
61
62          // Values can be left out.
63          (uint x, , uint y) = (4, 5, 6);
64
65          return (i, b, j, x, y);
66      }
67
68      // Cannot use map for neither input nor output
69
70      // Can use array for input
71      function arrayInput(uint[] memory _arr) public {}    infinite
72
73      // Can use array for output
74      uint[] public arr;
75
76      function arrayOutput() public view returns (uint[] memory) {
77          return arr;
78      }
79
80      function returnTwo()    472 gas
81          public
82          pure
83          returns (
84              int i,
85              bool b
86          )
87      {
88          i = -2;
89          b = true;
90      }
91  }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 9:

LEARNETH

Tutorials list | Syllabus

6. Visibility
9 / 19

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer

Show answer

Next

Well done! No errors.

Compile | visibility.sol | visibility_answer.sol

```
19      }
20
21      function publicFunc() public pure returns (string memory) {
22          return "public function called";
23      }
24
25      function externalFunc() external pure returns (string memory)
26          return "external function called";
27      }
28
29      string private privateVar = "my private variable";
30      string internal internalVar = "my internal variable";
31      string public publicVar = "my public variable";
32  }
33
34  contract Child is Base {
35
36      function testInternalFunc()    infinite gas
37          public
38          pure
39          override
40          returns (string memory)
41      {
42          return internalFunc();
43      }
44
45      function testInternalVar()    infinite gas
46          public
47          view
48          returns (string memory, string memory)
49      {
50          return (internalVar, publicVar);
51      }
52  }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 10:

LEARNETH

Tutorials list | Syllabus

7.1 Control Flow - If/Else
10 / 19

`else` statement (line 10) to return `2` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the foo function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternery operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer

Show answer

Next

Well done! No errors.

Compile | visibility.sol | ifElse.sol

```
1   // SPDX-License-Identifier: MIT
2   pragma solidity ^0.8.3;
3
4   contract IfElse {
5
6       function foo(uint x) public pure returns (uint) {    infinite
7           if (x < 10) {
8               return 0;
9           } else if (x < 20) {
10              return 1;
11          } else {
12              return 2;
13          }
14      }
15
16      // ⭐ Assignment
17      function evenCheck(uint x) public pure returns (bool) {
18          return (x % 2 == 0) ? true : false;
19      }
20  }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

>

Tutorial 11:

LEARNETH

Tutorials list | Syllabus

7.2 Control Flow - Loops
11 / 19

executed at least once, before checking on the condition.

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

1. Create a public `uint` state variable called count in the `Loop` contract.
2. At the end of the for loop, increment the count variable by 1.
3. Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer

Show answer

Next

Well done! No errors.

Compile | loops.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Loop {
5
6      uint public count;
7
8      function loop() public {    infinite gas
9
10         for (uint i = 0; i < 10; i++) {
11
12             if (i == 3) {
13                 continue;
14             }
15
16             if (i == 5) {
17                 break;
18             }
19
20             count += 1;
21         }
22
23         // ⭐ increment AFTER loop finishes
24         count += 5;
25     }
26 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
- ethers.js

Type the library name to see available commands.

>

Tutorial 12:

LEARNETH

Tutorials list | Syllabus

8.1 Data Structures - Arrays
12 / 19

We can use the `delete` operator to remove an element with a specific index from an array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

Assignment

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer

Show answer

Next

Well done! No errors.

Compile | loops.sol | arrays.sol

```
3
4  contract Array {
5
6      uint[] public arr;
7      uint[] public arr2 = [1, 2, 3];
8
9      // ⭐ Assignment
10     uint[3] public arr3 = [uint(0), 1, 2];
11
12     function get(uint i) public view returns (uint) {    infinite
13         return arr[i];
14     }
15
16     function push(uint i) public {    46820 gas
17         arr.push(i);
18     }
19
20     function pop() public {    29462 gas
21         arr.pop();
22     }
23
24     function getLength() public view returns (uint) {    2483 gas
25         return arr.length;
26     }
27
28     function remove(uint index) public {    7587 gas
29         delete arr[index];
30     }
31
32     // ⭐ Modified function
33     function getArr() public view returns (uint[3] memory) {
34         return arr3;
35     }
36 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
- ethers.js

Type the library name to see available commands.

>

Tutorial 13:

LEARNETH

Tutorials list | Syllabus

8.2 Data Structures - Mappings
13 / 19

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

Assignment

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer

Show answer

Next

Well done! No errors.

Compile | mappings.sol | mappings_answer.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Mapping {
5      mapping(address => uint) public balances;
6
7      function get(address _addr) public view returns (uint) {
8          return balances[_addr];
9      }
10
11     function set(address _addr) public {    25256 gas
12         balances[_addr] = _addr.balance;
13     }
14
15     function remove(address _addr) public {    5566 gas
16         delete balances[_addr];
17     }
18 }
19
20 contract NestedMapping {
21     mapping(address => mapping(uint => bool)) public nested;
22
23     function get(address _addr1, uint _i) public view returns (
24         return nested[_addr1][_i];
25     }
26
27     function set(address _addr1, uint _i, bool _boo) public {
28         nested[_addr1][_i] = _boo;
29     }
30
31     function remove(address _addr1, uint _i) public {    25045
32         delete nested[_addr1][_i];
33     }
34 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
- ethers.js

Type the library name to see available commands.

Tutorial 14:

LEARNETH

Tutorials list | Syllabus

8.3 Data Structures - Structs
14 / 19

There are different ways to initialize a struct.

Positional parameters: We can provide the name of the struct and the values of its members as parameters in parentheses (line 16).

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a structs' member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

Assignment

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer

Show answer

Next

Well done! No errors.

Compile | mappings.sol | mappings_answer.sol | structs.sol

```
8      }
9
10     Todo[] public todos;
11
12     function create(string memory _text) public {    infinite gas
13         todos.push(Todo(_text, false));
14         todos.push(Todo({text: _text, completed: false}));
15
16         Todo memory todo;
17         todo.text = _text;
18
19         todos.push(todo);
20     }
21
22     function get(uint _index) public view returns (string memory
23         Todo storage todo = todos[_index];
24         return (todo.text, todo.completed);
25     }
26
27     function update(uint _index, string memory _text) public {
28         Todo storage todo = todos[_index];
29         todo.text = _text;
30     }
31
32     function toggleCompleted(uint _index) public {    28995 gas
33         Todo storage todo = todos[_index];
34         todo.completed = !todo.completed;
35     }
36
37     // ⭐ Assignment
38     function remove(uint _index) public {    infinite gas
39         delete todos[_index];
40     }
41 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
- ethers.js

Type the library name to see available commands.

Tutorial 15:

LEARNETH

Tutorials list | Syllabus

8.4 Data Structures - Enums
15 / 19

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

Assignment

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.
2. Initialize the variable `sizes` of the enum type `Size`.
3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer

Show answer

Next

Well done! No errors.

Compile | igs_answer.sol | structs.sol | enums.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Enum {
5
6      // ⭐ Assignment
7      enum Size { S, M, L }
8
9      Size public sizes;
10
11     function getSize() public view returns (Size) {    2538 gas
12         return sizes;
13     }
14 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

```
- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
• ethers.js

Type the library name to see available commands.
>
```

Tutorial 16:

LEARNETH

Tutorials list | Syllabus

9. Data Locations
16 / 19

contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

Assignment

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myMemStruct2` with the data location *memory* inside the `function f` and assign it the value of `myMemStruct`. Change the value of the `myMemStruct2` member `foo` to 1.
3. Create a new struct `myMemStruct3` with the data location *memory* inside the `function f` and assign it the value of `myStruct`. Change the value of the `myMemStruct3` member `foo` to 3.
4. Let the function f return `myStruct`, `myMemStruct2`, and `myMemStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer

Show answer

Next

Well done! No errors.

Compile | structs.sol | enums.sol | dataLocations.sol

```
10
11     mapping(address => MyStruct) public myStructs;
12
13     function f()    infinite gas
14         public
15         returns (
16             MyStruct memory,
17             MyStruct memory,
18             MyStruct memory
19         )
20     {
21         // Storage reference
22         MyStruct storage myStruct = myStructs[msg.sender];
23
24         // Change storage struct
25         myStruct.foo = 4;
26
27         // Memory struct
28         MyStruct memory myMemStruct = MyStruct({
29             foo: 0,
30             text: "memory"
31         });
32
33         // Memory to memory (reference)
34         MyStruct memory myMemStruct2 = myMemStruct;
35         myMemStruct2.foo = 1;
36
37         // Storage to memory (copy)
38         MyStruct memory myMemStruct3 = myStruct;
39         myMemStruct3.foo = 3;
40
41         return (myStruct, myMemStruct2, myMemStruct3);
42     }
43 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

```
- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:
• ethers.js

Type the library name to see available commands.
>
```

Tutorial 17:

LEARNETH

Tutorials list | Syllabus

10.1 Transactions - Ether and Wei
17 / 19

`wei`

Wei is the smallest subunit of *Ether*, named after the cryptographer Wei Dai. *Ether* numbers without a suffix are treated as `wei` (line 7).

`gwei`

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

`ether`

One `ether` is equal to 1,000,000,000,000,000,000 (10^18) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

Assignment

1. Create a `public` `uint` called `oneGWei` and set it to 1 `gwei`.
2. Create a `public` `bool` called `isOneGWei` and set it to the result of a comparison operation between 1 gwei and 10^9.

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer

Show answer

Next

Well done! No errors.

Compile | dataLocations.sol | etherAndWei.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract EtherUnits {
5
6      uint public oneWei = 1 wei;
7      uint public oneEther = 1 ether;
8
9      // ⭐ Assignment (correct casing)
10     uint public oneGwei = 1 gwei;
11
12     bool public isOneGwei = (1 gwei == 10**9);
13 }
```

Explain contract | AI copilot

Listen on all transactions | Filter with transaction hash or ad...

- Right-click on a JavaScript file in the file explorer and then click `Run`

The following libraries are accessible:

- ethers.js

Type the library name to see available commands.

Tutorial 18:

LEARNETH

Tutorials list | Syllabus

10.2 Transactions - Gas and Gas Price
18 / 19

going through faster; transactions with very low *gas prices* might not go through at all.

When sending a transaction, the sender has to pay the *gas* fee (gas_price * gas) upon execution of the transaction. If *gas* is left over after the execution is completed, the sender gets refunded.

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of *gas* before being completed, reverting any changes being made. In this case, the *gas* was consumed and can't be refunded.

Learn more about *gas* on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new `public` state variable in the `Gas` contract called `cost` of the type `uint`. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer | Show answer

Next

Well done! No errors.

Compile | gasAndGasPrice.sol

```
1  // SPDX-License-Identifier: MIT
2  pragma solidity ^0.8.3;
3
4  contract Gas {
5      uint public i = 0;
6      uint public cost = 170367;
7
8      function forever() public {    infinite gas
9          while (true) {
10             i += 1;
11         }
12     }
13 }
```

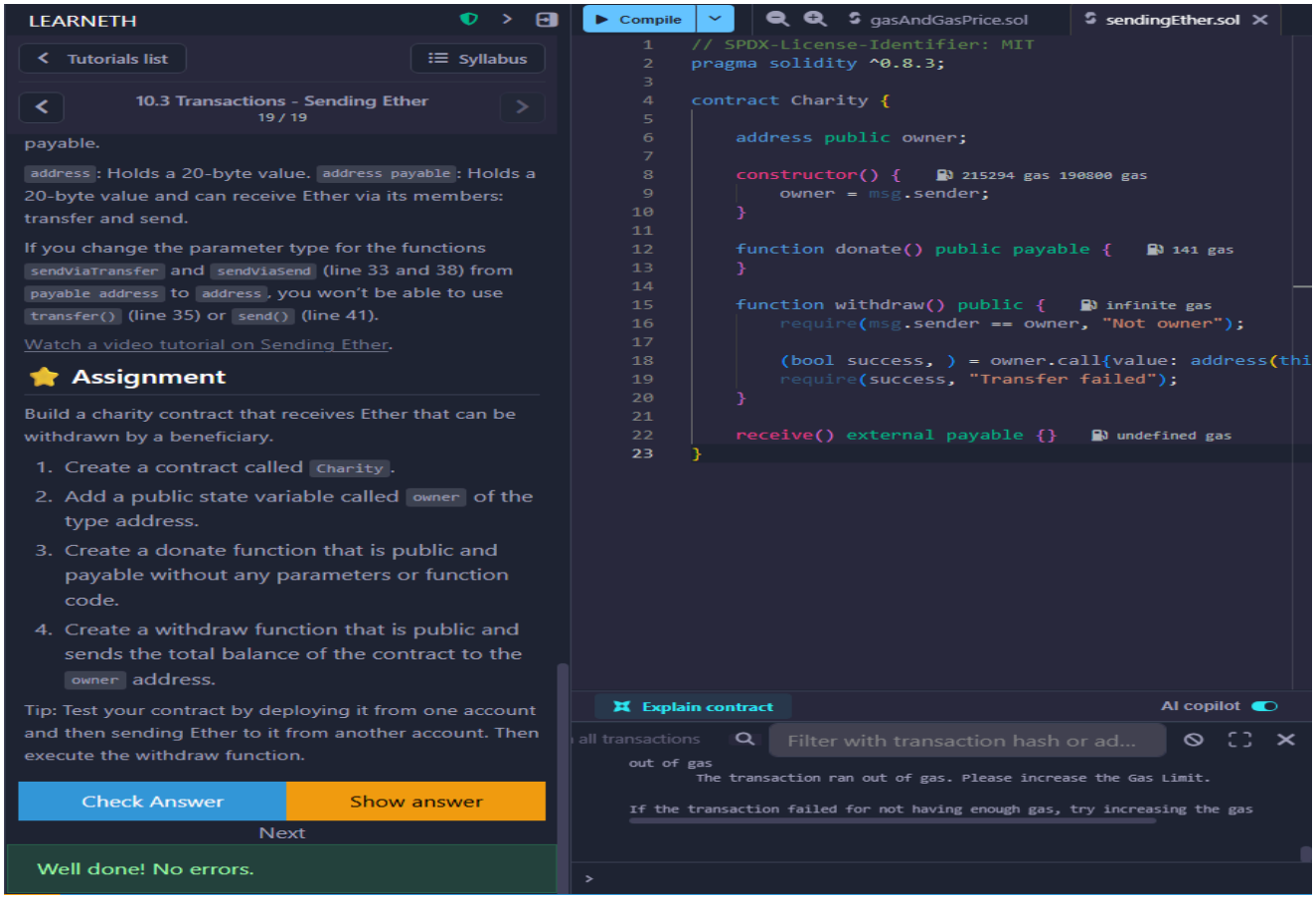
Explain contract | AI copilot

all transactions | Filter with transaction hash or ad...

out of gas
The transaction ran out of gas. Please increase the Gas Limit.

If the transaction failed for not having enough gas, try increasing the gas

Tutorial 19:



Conclusion:

This experiment helped in understanding Solidity and smart contract development in a practical way using Remix IDE. By working through different assignments, important concepts like variables, functions, modifiers, visibility, control flow, data structures (arrays, mappings, structs, enums), data locations, and Ether transactions were clearly understood through implementation rather than just theory.

Deploying and testing contracts on the Remix VM also gave insight into how gas, Ether units, and transaction costs work in real scenarios. Overall, this experiment built a solid foundation in writing, compiling, and interacting with smart contracts, and improved confidence in developing basic decentralized applications on the Ethereum blockchain.